

Signals, Computed & Effect

Conocimientos previos:

- Módulos en Angular (En otro documento)
- Funciones Flecha (En anexo)
- Operador de dispersión (En anexo)

Signal

Es una **fuentes de estado reactivo**. Almacena un valor y permite leerlo o actualizarlo sin necesidad de ir manualmente a alterar el valor en pantalla.

Lo que ocurre por detrás es que **lee el valor de la variable cuando cambia** y va al **contenedor de html** donde se la está usando para **cambiar el valor que vemos en la pantalla**.

Se define de la siguiente manera (Imagen 1):

```
signal<type>(init_value)
```

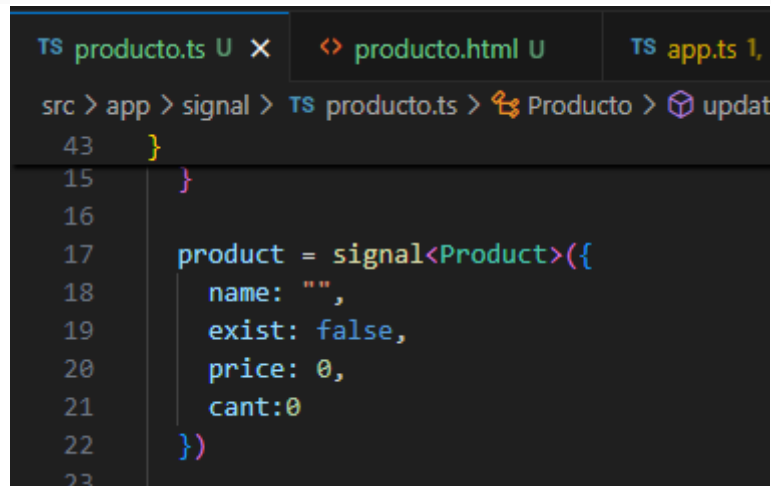


Imagen 1 - Inicialización de Signal

En el html no se utiliza como una variable normal, anteriormente usábamos `{{ nombre_variable }}`, pero técnicamente las **signals no son variables**, son funciones, por lo que en el html tenemos que reflejar esto usando los **parentesis** `{{ signal_name() }}` (Imagen 2). Ya solamente con esto la **signal va a detectar el cambio de valor, va a ese elemento del html, y actualiza el valor en pantalla**.

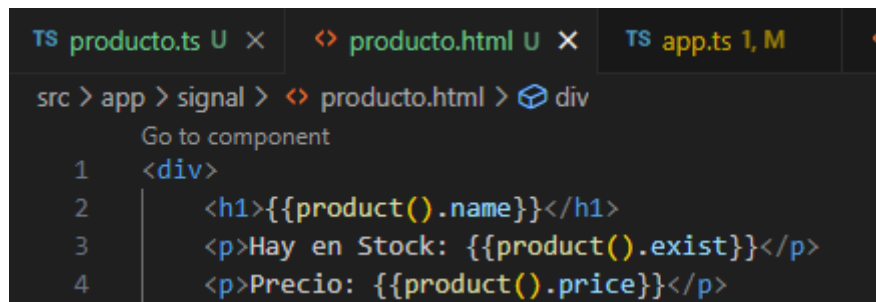


Imagen 2 - Signal en un HTML

Nótese que cuando estamos trabajando con un objeto, luego de los paréntesis podemos decir el atributo de ese objeto en particular.

Se puede **actualizar o asignar** un valor a una signal mediante las funciones que esta tiene definidas por defecto. Con **update actualizamos el valor** (Imagen 3 e Imágen 4), mientras que con **set asignamos un valor nuevo** (Imagen 5).

```
TS producto.ts U x <> producto.html U TS app.t
src > app > signal > TS producto.ts > Producto > ch
53 }
23
24 updateProductPrice() {
25   this.product.update(p => {
26     return {
27       ...p,
28       price: p.price + 100
29     }
30   })
31 }
32 }
```

Imagen 3 - Update ejemplo

```
TS producto.ts U x <> producto.html U TS app.
src > app > signal > TS producto.ts > Producto > to
53 }
33
34 toggleExist() {
35   this.product.update(p => {
36     return {
37       ...p,
38       exist: false
39     }
40   })
41 }
42 }
```

Imagen 4 - Update ejemplo

```
TS producto.ts U x <> producto.html U x TS .
src > app > signal > TS producto.ts > Producto > f
11 export class Producto {
43
44   changeProduct() {
45     this.product.set({
46       name: 'Fernet',
47       exist: true,
48       price: 5200,
49       cant: 0
50     })
51   }
52 }
53 }
54 }
```

Imagen 5 - Set Ejemplo

El flujo sería algo similar a esto:

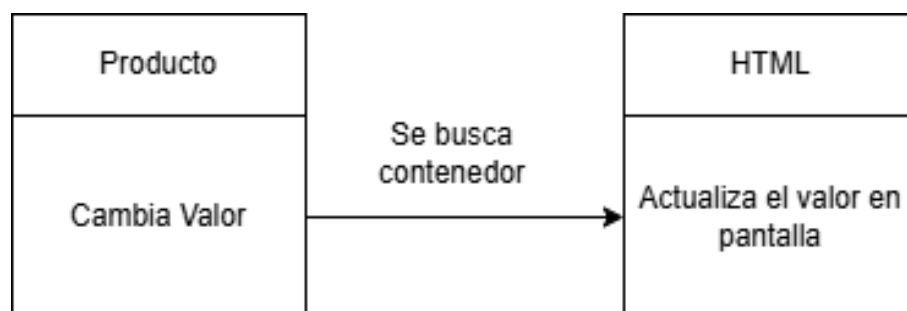


Imagen 6 - Flujo se una signal

Computed

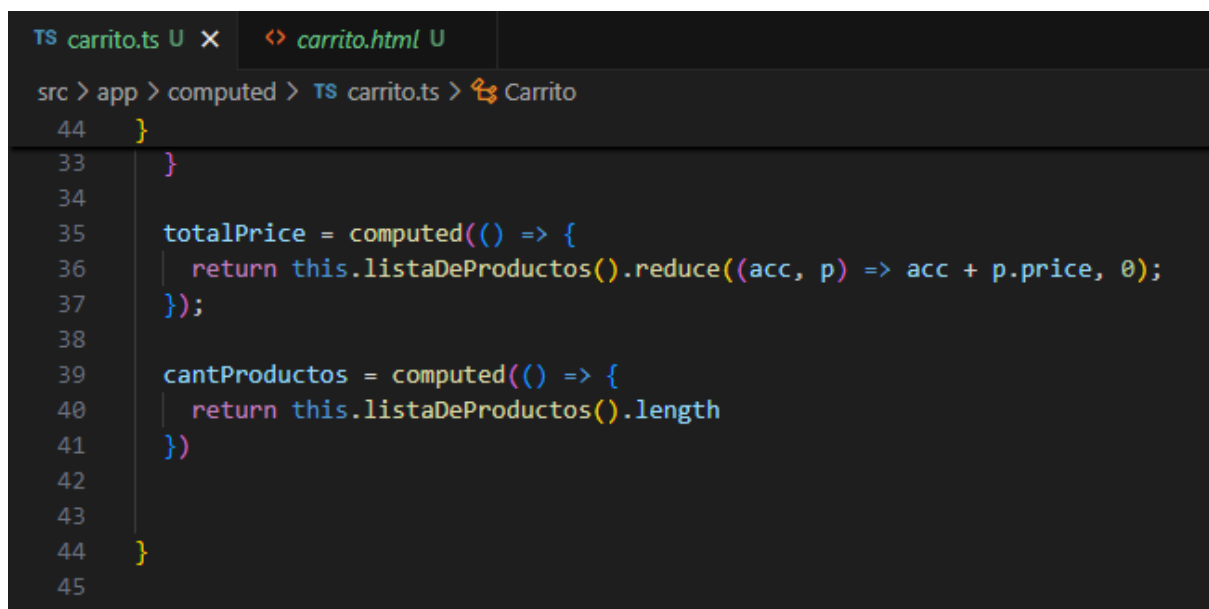
Es un **valor derivado automáticamente** de uno o más **signal**. Se **recalcula cuando alguna señal asociada a este valor cambia**.

Por ejemplo, si tenemos cantidad y precio de un producto, ambos siendo signals, podemos asignarle al computed que tome el valor del producto y lo multiplique por la cantidad, de forma automática.

Se define de la siguiente manera:

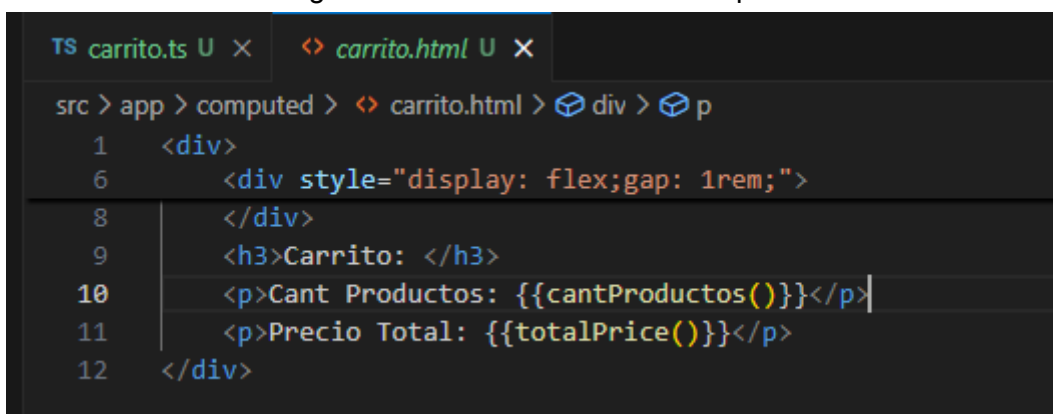
```
computed(() => { return process })
```

Donde la función flecha es el **proceso** que realizará cuando **detecte el cambio**. **Muy importante, esta función debe retornar un valor** (Imagen 7). En html se implementa de la misma manera que la signal, **llamado a la función correspondiente** (Imagen 8)



```
TS carrito.ts U X  <> carrito.html U
src > app > computed > TS carrito.ts > Carrito
44   }
33   }
34
35   totalPrice = computed(() => {
36     return this.listaDeProductos().reduce((acc, p) => acc + p.price, 0);
37   });
38
39   cantProductos = computed(() => {
40     return this.listaDeProductos().length
41   })
42
43
44   }
45
```

Imagen 7 -Definición de valores computed



```
TS carrito.ts U X  <> carrito.html U X
src > app > computed > <> carrito.html > div > p
1   <div>
6     <div style="display: flex;gap: 1rem;">
8       </div>
9       <h3>Carrito: </h3>
10      <p>Cant Productos: {{cantProductos()}}</p>
11      <p>Precio Total: {{totalPrice()}}</p>
12    </div>
```

Imagen 8 -Definición de valores computed

El flujo sería algo similar a esto:

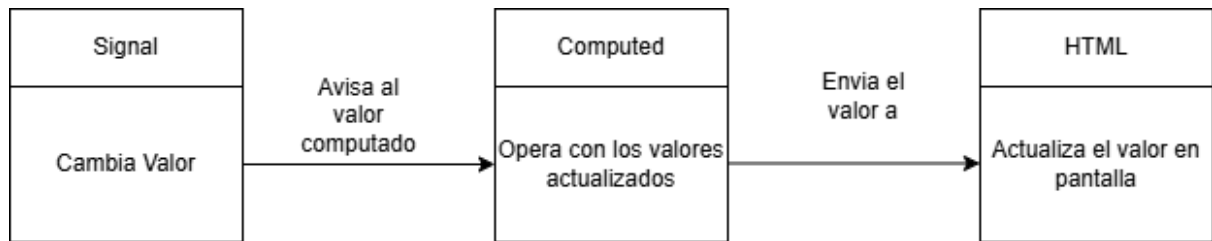


Imagen 9 - Flujo Computed

Effects

Se usa para **ejecutar efectos secundarios** (como logs, API calls, DOM updates) cuando cambian señales utilizadas dentro. Es muy **similar a los valores computados**, solo que **los effect no retornan un valor**, su utilidad no radica en modificar los valores en pantalla sino que están más orientados a ejecución de procesos por detrás.

Se define de la siguiente manera:

```
effect(() => {{ process }})
```

Donde la función **flecha es el proceso que va a realizar**. Nótese que en este caso no tenemos un **return** (Imagen 10). En este caso no hay que hacer nada en el **html**, ya que todo ocurre por atrás (Imagen 11).



Imagen 10 - Definición de un Effect

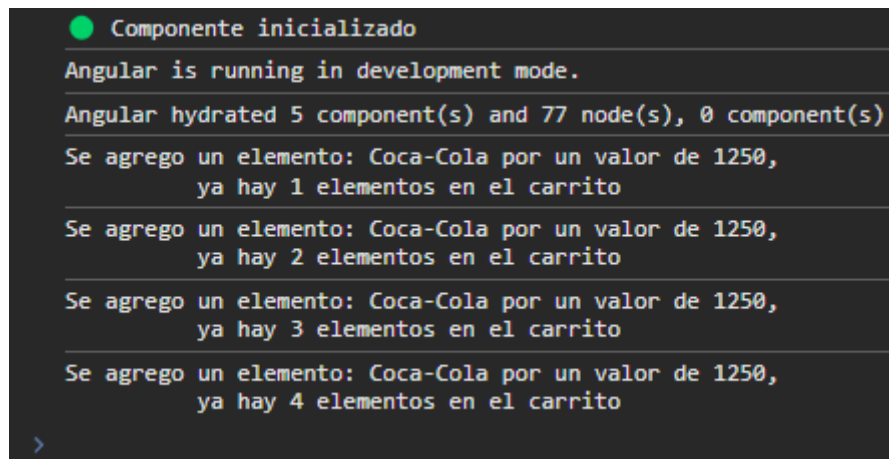


Imagen 11 - Consola con Logs

El flujo sería algo similar a esto:

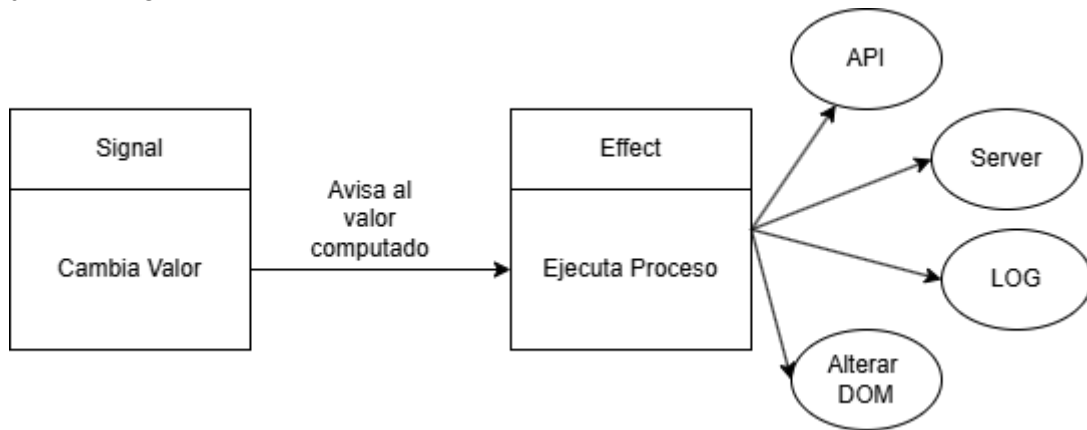


Imagen 12 - Flujo Effect

Métodos del ciclo de vida

Te permiten ejecutar código en momentos clave del proceso de creación y destrucción del componente. Estos métodos vienen predefinidos y ocurren de forma automática sin necesidad de ser llamados.

Principales métodos del ciclo de vida:

Método	Cuándo se ejecuta
<code>ngOnInit()</code>	Al inicializar el componente (una vez, después del constructor). Ideal para cargar datos.
<code>ngOnDestroy()</code>	Justo antes de destruir el componente. Útil para limpiar subscripciones, timers, etc.
<code>ngOnChanges()</code>	Cada vez que cambian los inputs (<code>@Input</code>). Se ejecuta múltiples veces.
<code>ngAfterViewInit()</code>	Después de que se inicializan las vistas hijas (child components).
<code>ngAfterViewChecked()</code>	Luego de cada verificación de vista (detección de cambios).
<code>ngAfterContentInit()</code>	Luego de proyectar contenido en <code><ng-content></code> .
<code>ngAfterContentChecked()</code>	Cada vez que se verifica ese contenido proyectado.

Ejemplo Simple

```
export class MiComponente implements OnInit, OnDestroy {
  ngOnInit() {
    console.log('Componente inicializado');
  }

  ngOnDestroy() {
    console.log('Componente destruido');
  }
}
```


Anexo

Función Flecha

Es una forma moderna de escribir funciones en JavaScript (y TypeScript). Ejemplos:

```
// Tradicional
const cuadrado = function(x) {
  return x * x;
};

// Arrow function
const cuadrado = (x) => x * x;
```

Características importantes:

- Si tiene un solo parámetro, podés omitir los paréntesis: `x => x * x`
Si el cuerpo tiene más de una línea, necesitás llaves y `return`:

```
const sumar = (a, b) => {
  const resultado = a + b;
  return resultado;
};
```

- **No tiene su propio `this`:** Esto es útil en callbacks, porque no cambia el contexto como una función normal.

Operador de Dispersión

Permite **copiar o combinar** objetos/arrays sin modificar el original (inmutabilidad).

En Arrays

```
const a = [1, 2, 3];  
const b = [...a, 4]; // [1, 2, 3, 4]
```

También sirve para **copiar**:

```
const copia = [...a]; // misma lista, nueva referencia
```

En Objetos

```
const persona = { nombre: "Alan", edad: 25 };  
const actualizada = { ...persona, edad: 27 };  
// copia persona, pero sobrescribe edad
```

Este patrón es común en Angular Signals o Redux:

```
this.product.set(p => ({  
  ...p,  
  exist: !p.exist  
}));
```

Esto **clona** el objeto **p** y solo cambia el campo **exist**.